

I grew up with a game called “Concentration”; my mom said I would play for hours along with my puzzles. It was a fun game using only a typical deck of cards.

During the days when Flash was part of our curriculum, students needed to learn ActionScript 3.0. The best and most successful activities were game related. I miss those Flash days!

The game you are about to create uses only JavaScript, some ES6 and basic DOM concepts.

Goals

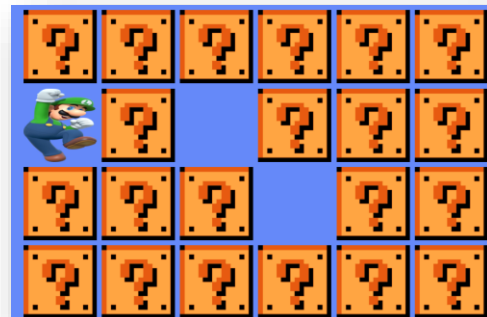
The game has a grid of 24 face-down cards; the card faces consist of pairs of matches. Clicking on cards will flip them over, revealing the value. When two are chosen, if it results in a match, both cards disappear. If the result is not a match, the cards will flip back over to face-down. The game should be different every time the game is refreshed.

Planning

As with any major code problem, it is best to itemize the actual steps. Some developers use flowcharts, others pseudocode; regardless of the method, it helps to break down the code into manageable tasks.

Game Objectives

- Display 12 cards
- Duplicate the cards to have 2 sets of 12
- Randomize the display of cards
- Add selected style for selected cards
- Only allow two cards to be selected at a time
- Determine if two selected cards are a match and hide them
- Reset guess count after 2
- Add delay to selections
- Show back of card initially and flip on select
- Finished game!



Starting Point

Launch the start files for this activity. The file is organized with the following:

- css > style.css (uses a flex grid for the cards)
- images > (images used for the cards)
- js
- index.html

Take time to explore the css file so that you have an understanding of the code.

Create a new JavaScript file and then save it as script.js.

Use the image as a guide to create the code that will display the cards.

First, establish an array for all of the images.

```
1  'use strict';
2
3  let cardsArray = [{
4    'name': 'shell',
5    'images': 'images/blueshell.png'
6  }, {
7    'name': 'star',
8    'images': 'images/star.png'
9  }, {
10   'name': 'bobomb',
11   'images': 'images/bobomb.png'
12 }, {
13   'name': 'mario',
14   'images': 'images/mario.png'
15 }, {
16   'name': 'luigi',
17   'images': 'images/luigi.png'
18 }, {
19   'name': 'peach',
20   'images': 'images/peach.png'
21 }, {
22   'name': 'lup',
23   'images': 'images/lup.png'
24 }, {
25   'name': 'mushroom',
26   'images': 'images/mushroom.png'
27 }, {
28   'name': 'thwomp',
29   'images': 'images/thwomp.png'
30 }, {
31   'name': 'bulletbill',
32   'images': 'images/bulletbill.png'
33 }, {
34   'name': 'coin',
35   'images': 'images/coin.png'
36 }, {
37   'name': 'goomba',
38   'images': 'images/goomba.png'
39 }];
```

The cards are all set, now it's time to display them and randomize the display. The result will be something like the following image.



```
38   'images': 'images/goomba.png'
39   });
40   // Duplicate array to create a match for each card
41   // Randomize the cards
42   let gameGrid = cardsArray.concat(cardsArray);
43   gameGrid.sort(() => 0.5 - Math.random());
44
45   // capture the div with an id of game
46   const game = document.getElementById('game');
47   // create a section with a class of grid
48   const grid = document.createElement('section');
49   grid.setAttribute('class', 'grid');
50   // append the grid section to the game div
51   game.appendChild(grid);
52
53   gameGrid.forEach(item =>
54   {
55     // Create a div
56     const card = document.createElement('div');
57     // Apply a card class to that div
58     card.classList.add('card');
59     // Set the data-name attribute of the div to the cardsArray name
60     card.dataset.name = item.name;
61     // Apply the background image of the div to the cardsArray image
62     card.style.backgroundImage = `url(${item.images})`;
63     // Append the div to the grid section
64     grid.appendChild(card);
65   });
66
```

Continue to code the script by adding an event listener to the entire grid. Anytime an element is clicked, the selected class will be applied. Also, only two selections at a time are permitted.

Create a new variable.

```
43 gameGrid.sort(() => 0.5 - Math.random());
44
45 let count = 0;
46
47 // capture the div with an id of game
```

```
67 });
68
69 // Add event listener to grid
70 grid.addEventListener('click', function(event)
71 {
72   // The event target is our clicked item
73   let clicked = event.target
74
75   // Do not allow the grid section itself to be selected; only select divs inside the grid
76   if (clicked.nodeName === 'SECTION')
77   {
78     return
79   }
80   // Allow only 2 cards to be selected at a time
81   if (count < 2)
82   {
83     count++;
84     clicked.classList.add('selected');
85   }
86 });
87
```

In order to test, I added a property to the .selected class.

```
.selected
{
  -webkit-transform: rotateY(180deg);
  transform: rotateY(180deg);
  border: 5px solid purple;
}
```



Continue to code to see if the two selected cards are a match and then hide them. Update the code to add variables.

```
47 // variables
48 let firstGuess = '';
49 let secondGuess = '';
50 let count = 0;
```

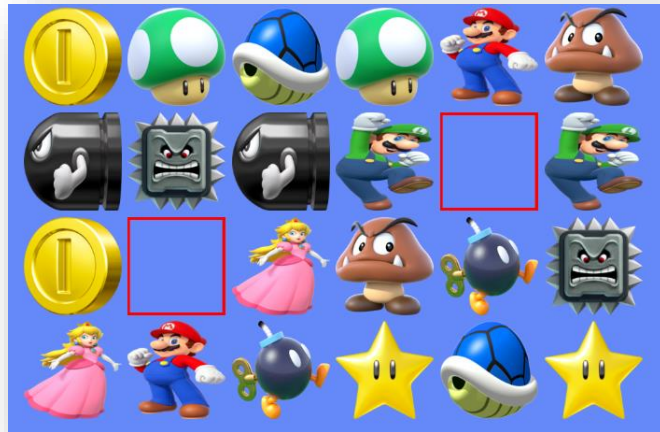
Create a function for matching elements. First, I modified the css for testing purposes.

```
78 .match
79 {
80   border: 5px solid red;
81   background: #6589F9 !important;
82 }
83
```

Modify the code to make sure of the updates, use the image as a guide.

```
72 // Add event listener to grid
73 grid.addEventListener('click', function(event)
74 {
75 // The event target is our clicked item
76 let clicked = event.target
77
78 // Do not allow the grid section itself to be selected; only select divs inside the grid
79 if (clicked.nodeName === 'SECTION')
80 {
81 return
82 }
83 // Allow only 2 cards to be selected at a time
84 if (count < 2)
85 {
86 count++
87 if (count === 1)
88 {
89 // Assign first guess
90 firstGuess = clicked.dataset.name
91 clicked.classList.add('selected')
92 }
93 else
94 {
95 // Assign second guess
96 secondGuess = clicked.dataset.name
97 clicked.classList.add('selected')
98 }
99 // If both guesses are not empty...
100 if (firstGuess !== '' && secondGuess !== '')
101 {
102 // and the first guess matches the second match...
103 if (firstGuess === secondGuess)
104 {
105 // run the match function
106 match()
107 }
108 }
109 }
110 })
111 // Add match CSS
112 const match = () =>
113 {
114 var selected = document.querySelectorAll('.selected')
115 selected.forEach(card =>
116 {
117 card.classList.add('match')
118 })
119 }
```

If you select 2 cards that match, the css is applied.



Did you select the same element twice? If so, it should have done the following.



Update the code to prevent the selection of the same element, reset the count after 2, and add a delay to the selections.

```
47 // variables
48 let firstGuess = '';
49 let secondGuess = '';
50 let count = 0;
51 let previousTarget = null;
52 let delay = 1200;
53
```

Basically, the image below is the complete js code.


```
1  'use strict';
2
3  let cardsArray = [{
4    'name': 'shell',
5    'images': 'images/blueshell.png'
6  }, {
7    'name': 'star',
8    'images': 'images/star.png'
9  }, {
10   'name': 'bobomb',
11   'images': 'images/bobomb.png'
12 }, {
13   'name': 'mario',
14   'images': 'images/mario.png'
15 }, {
16   'name': 'luigi',
17   'images': 'images/luigi.png'
18 }, {
19   'name': 'peach',
20   'images': 'images/peach.png'
21 }, {
22   'name': 'lup',
23   'images': 'images/lup.png'
24 }, {
25   'name': 'mushroom',
26   'images': 'images/mushroom.png'
27 }, {
28   'name': 'thwomp',
29   'images': 'images/thwomp.png'
30 }, {
31   'name': 'bulletbill',
32   'images': 'images/bulletbill.png'
33 }, {
34   'name': 'coin',
35   'images': 'images/coin.png'
36 }, {
37   'name': 'goomba',
38   'images': 'images/goomba.png'
39 }];
40 // Duplicate array to create a match for each card
41 // Randomize the cards
42 let gameGrid = cardsArray.concat(cardsArray).sort(function ()
43 {
44   return 0.5 - Math.random();
45 });
46
```

```
46
47 // variables
48 let firstGuess = '';
49 let secondGuess = '';
50 let count = 0;
51 let previousTarget = null;
52 let delay = 1200;
53
54
55 // capture the div with an id of game
56 const game = document.getElementById('game');
57 // create a section with a class of grid
58 const grid = document.createElement('section');
59 grid.setAttribute('class', 'grid');
60 // append the grid section to the game div
61 game.appendChild(grid);
62
63 gameGrid.forEach(function (item)
64 {
65     let name = item.name,
66         img = item.images;
67
68
69     let card = document.createElement('div');
70     card.classList.add('card');
71     card.dataset.name = name;
72
73     let front = document.createElement('div');
74     front.classList.add('front');
75
76     let back = document.createElement('div');
77     back.classList.add('back');
78     back.style.backgroundImage = 'url(' + img + ')';
79
80     grid.appendChild(card);
81     card.appendChild(front);
82     card.appendChild(back);
83 });
84
```

```
107
108 grid.addEventListener('click', function (event)
109 {
110
111     let clicked = event.target;
112
113     if (clicked.nodeName === 'SECTION' || clicked === previousTarget || clicked.parentNode.classList.contains('selected') || clicked.parentNode.classList.contains('match'))
114     {
115         return;
116     }
117
118     if (count < 2)
119     {
120         count++;
121         if (count === 1)
122         {
123             firstGuess = clicked.parentNode.dataset.name;
124             console.log(firstGuess);
125             clicked.parentNode.classList.add('selected');
126         } else
127         {
128             secondGuess = clicked.parentNode.dataset.name;
129             console.log(secondGuess);
130             clicked.parentNode.classList.add('selected');
131         }
132
133         if (firstGuess && secondGuess)
134         {
135             if (firstGuess === secondGuess)
136             {
137                 setTimeout(match, delay);
138             }
139             setTimeout(resetGuesses, delay);
140         }
141         previousTarget = clicked;
142     }
143 });
144
145
```

I hope you enjoyed working through this activity. Why not let your family members play? How about modifying the code so that it provides a message for winning the game, counting the attempts, use different images, etc.